EEL 6764 – Computer Architecture
Final Project Report

Cache Hierarchy Sensitivity and Bottleneck Attribution for GEMM Using gem5 O3CPU Simulation

Team Members

- Ariel Gomez Garcia
- Jason Olefson
- Ryan Carbone
- Sebastien Ghent

Team Name: Cache Me If You Can
Institution: Bellini College of Artificial Intelligence, Cybersecurity and Computing, University of South Florida

Date: April 22, 2026

Contents

Project Title 1

Team Members 1

1 Project Overview 3
Problem Statement . . . 3
Importance . . . 3
Research Question and Hypothesis . . . 3
Project Goals . . . 3

2 Workload and Simulation Infrastructure 4
2.1 Simulator Configuration . . . 4
2.2 Workloads . . . 4
2.3 Metrics Collected . . . 5
2.4 Experiment Matrix . . . 5
2.5 Automation and Reproducibility . . . 5

3 Simulation-Driven Bottleneck Analysis 6
3.1 L1D Sensitivity at N=128 (L2 = 256 kB Fixed) . . . 6
3.2 L2 Sensitivity at N=128 (L1D = 32 kB Fixed) . . . 7
3.3 Working-Set Scaling (N=32 to 256) . . . 8
3.4 Bottleneck Attribution: Where Are the Cycles Going? . . . 9

4 Design Exploration and Optimization 10
4.1 Architectural Change Explored . . . 10
4.2 Performance Improvements . . . 10
4.3 Trade-offs: Performance vs. Capacity, Energy, and Complexity . . . 11

5 Key Insights and Takeaways 12

6 Limitations and Future Work 13
Limitations . . . 13
Future Work . . . 13

7 References 14

8 AI Usage Disclosure 14

Appendix A: Full Results Table 15

Appendix B: Reproducibility 15

Appendix C: Figure-to-Claim Map 16

1. Project Overview

Problem Statement

Dense matrix–matrix multiplication (GEMM) is a foundational kernel in scientific computing, machine learning, and signal processing. Although the kernel is simple to express, its performance on modern out-of-order processors is dominated by the behavior of the cache hierarchy. Understanding *where* in the memory subsystem a given GEMM configuration becomes bottlenecked—and *how much* it costs (in silicon area, energy, and design complexity) to relieve that bottleneck—is a prerequisite for principled microarchitectural design.

Importance

GEMM's streaming access pattern stresses every level of the cache hierarchy. A systematic sensitivity study exposes which level is the first-order limiter, how that limiter shifts as the working set grows, and what the marginal benefit of additional cache capacity actually is in practice. These insights directly inform cost/benefit decisions about SRAM budgets in real processor designs.

Research Question and Hypothesis

Research question: For an ijk-order GEMM kernel running on a gem5 `O3CPU`, which level of the cache hierarchy is the dominant performance bottleneck, how does that bottleneck shift with problem size N, and what is the cost–benefit profile of relieving it via larger L2?

Hypothesis: Because the ijk kernel streams through three $N \times N$ matrices, L1D miss rate will remain high independently of L1D size, and L2 capacity (rather than L1D capacity) will dominate achieved IPC once the working set exceeds L2. Specifically, we expect a clear performance "knee" at the L2 size that first contains the full $3N^2 \times 8$ byte working set.

Project Goals

Using the gem5 full-system simulator, we aim to:

(i) identify the bottlenecks in the memory hierarchy for an ijk-order GEMM kernel;
(ii) characterize the sensitivity of performance and cache behavior to changes in L1 and L2 cache sizes;
(iii) propose and evaluate a mitigation (larger L2 cache) and quantify its cost–benefit tradeoff.

2. Workload and Simulation Infrastructure

2.1 Simulator Configuration

All experiments were conducted with the **gem5 simulator** using the `O3CPU` (detailed out-of-order) processor model. Table 1 summarizes the fixed baseline parameters shared across experiment sets unless explicitly varied.

Table 1: Baseline simulation configuration.

Parameter	**Value**
Simulator	gem5
CPU model	`O3CPU` (detailed out-of-order)
L1 data cache (L1D)	32 kB
L1 instruction cache (L1I)	32 kB
L2 unified cache	256 kB
Memory system	gem5 default DRAM model
GEMM kernel loop order	ijk

Modifications. No source-level modifications were made to the gem5 simulator. All experimental variation is driven through configuration parameters (cache sizes, problem size N) surfaced in our experiment manifests.

2.2 Workloads

Benchmark. The workload binary is `gemm_x86`, a dense matrix–matrix multiplication kernel compiled for an ijk-loop-order implementation operating on double-precision floating-point matrices. This is a classic textbook GEMM loop structure in which the innermost loop streams along the column of B and accumulates into a single element of C.

Input sizes. The matrix dimension N was swept across $N \in \{32, 64, 128, 192, 256\}$. The total working-set footprint is $3 \times N^2 \times 8$ bytes (three $N \times N$ double-precision matrices), yielding footprints ranging from 24 kB at N=32 up to 1.5 MB at N=256.

Justification. This range was chosen deliberately to straddle the capacities of L1D (32 kB), L2 (256 kB baseline), and a range of hypothetical larger L2 configurations (up to 2 MB). This allows us to observe the transition from compute-efficient (working set fits in L1D) through L2-resident to memory-system-limited behavior within a single kernel.

2.3 Metrics Collected

For every run, the following metrics were extracted from gem5 output:

- **IPC** — instructions per cycle.
- **CPI** — cycles per instruction ($= 1/\text{IPC}$).
- **L1D MPKI** — L1 data-cache misses per kilo-instruction.
- **L2 MPKI** — L2 cache misses per kilo-instruction.
- **simSeconds** — simulated execution time.

L1I misses, branch mispredictions, and front-end stall cycles were monitored but are not reported as primary metrics because they were not material limiters in any tested configuration (see Section 3).

2.4 Experiment Matrix

Three experiment sets were executed, totaling 14 unique simulation configurations (10 initial matrix + 4 remediation, with one single-point resumption). The main matrix used manifest `experiments/final_o3_manifest.json` and completed as session `o3_run_20260421T184741Z` with 10 requested, 10 executed, and 0 failed runs.

1. **Cache sensitivity at** N=128**:**
   - L1D sweep: $\{16, 32, 64\}$ kB, with L2=256 kB fixed.
   - L2 sweep: $\{128, 256, 512\,\text{kB}, 1\,\text{MB}\}$, with L1D=32 kB fixed.
2. **Working-set scaling** at baseline caches: $N \in \{32, 64, 128, 192, 256\}$.
3. **L2 remediation at** N=256 (L1D=32 kB fixed): L2 $\in \{256\,\text{kB}, 512\,\text{kB}, 1\,\text{MB}, 2\,\text{MB}\}$.

2.5 Automation and Reproducibility

The automated workflow is: (1) edit the experiment list in `experiments/final_o3_manifest.json`; (2) run the driver (`O3CPU` is enforced by the scripts); (3) rebuild the consolidated CSV.

```
python3 scripts/run_o3_experiments.py \
        --manifest experiments/final_o3_manifest.json

python3 scripts/summarize_o3_results.py \
        --results_root results \
        --output_csv experiments/results_summary_o3.csv
```

Each run writes under `results/` in per-benchmark, per-tag subdirectories containing `run.log`, `command.sh`, `config_used.json`, and `run_record.json`. All tables in this report are drawn from O3 rows in `experiments/results_summary_o3.csv`. Representative *wall-clock* runtimes on our host were ≈184–230 s per N=128 configuration, ≈406 s for N=192, and ≈1307 s for N=256 at baseline caches.

3. Simulation-Driven Bottleneck Analysis

This section presents the quantitative evidence that identifies the dominant performance bottleneck: **L2 capacity**. We proceed in three steps: (1) fix the working set and sweep L1D to rule it out as the limiter, (2) fix L1D and sweep L2 to establish L2 as the first-order control knob, and (3) scale the working set to show how the bottleneck sharpens with problem size.

3.1 L1D Sensitivity at N=128 (L2 = 256 kB Fixed)

Table 2: L1D sensitivity at N=128, L2=256 kB.

L1D (kB)	IPC	L1D MPKI	L2 MPKI
16	1.619467	144.0296	0.2298
32	1.619324	143.8734	0.2380
64	1.619359	143.2164	0.2401

As shown in Table 2 and Figure 1, IPC is virtually constant across the tested L1D sizes, varying by less than 0.01%. L1D MPKI is extremely high in all three cases (≈143–144), which reflects the streaming nature of the *ijk* GEMM loop: the kernel accesses matrix elements that do not persist in L1D between iterations regardless of whether L1D is 16 kB or 64 kB. Critically, these L1D misses are absorbed by L2 (L2 MPKI remains below 0.25 in all cases), confirming that **L1D capacity is not the first-order performance limiter** for N=128 when L2 is adequately sized.

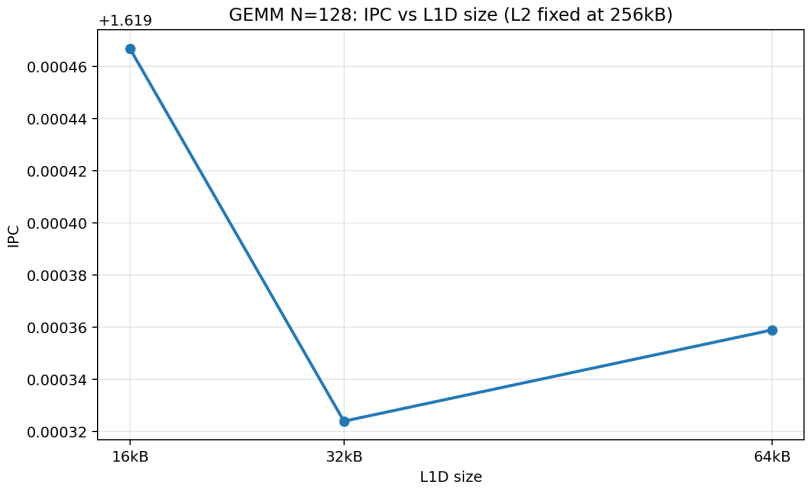


Figure 1: IPC vs. L1D size at N=128 (`O3CPU`). Per-run L1D MPKI values appear in Table 2.

3.2 L2 Sensitivity at N=128 (L1D = 32 kB Fixed)

Table 3: L2 sensitivity at N=128, L1D=32 kB.

L2 size	IPC	CPI	L1D MPKI	L2 MPKI
128 kB	0.724 949	1.379 407	124.0400	31.7941
256 kB	1.619 324	0.617 542	143.8734	0.2380
512 kB	1.618 838	0.617 727	143.4008	0.0735
1 MB	1.620 607	0.617 053	143.3411	0.0537

Reducing L2 cache size from 256 kB to 128 kB causes IPC to fall by more than **2.2**× (from 1.619 to 0.725) while L2 MPKI rises by over **130**× (from 0.24 to 31.79). CPI more than doubles from 0.62 to 1.38. Conversely, increasing L2 above 256 kB yields no meaningful improvement—IPC remains within 0.1% across {256 kB, 512 kB, 1 MB}—indicating that the working set at N=128 fits within 256 kB.

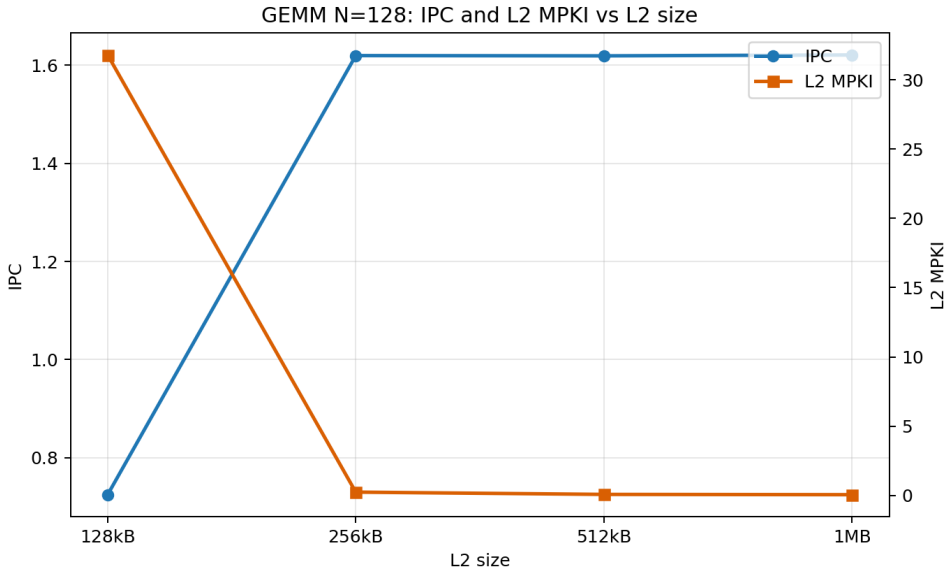


Figure 2: IPC and L2 MPKI vs. L2 size at N=128 (`O3CPU`).

3.3 Working-Set Scaling (N=32 to 256)

Holding caches at their baseline sizes (L1D=32 kB, L2=256 kB), we swept the matrix dimension N.

Table 4: Performance scaling with problem size (baseline caches: L1D=32 kB, L2=256 kB).

N	Footprint	IPC	CPI	L1D MPKI	L2 MPKI	simSeconds
32	24 kB	1.442 989	0.693 006	1.5073	2.3896	0.000 150
64	96 kB	2.126 024	0.470 362	43.9233	0.5210	0.000 681
128	384 kB	1.619 324	0.617 542	143.8734	0.2380	0.006 575
192	864 kB	1.391 842	0.718 472	151.8707	17.2738	0.025 106
256	1.5 MB	0.451 756	2.213 586	141.9651	137.6844	0.180 780

Figure 3 and Table 4 reveal a clear transition from compute-efficient to memory-system-limited behavior:

- At N=32, the 24 kB working set fits within L1D. IPC is moderate (1.44) and both L1D and L2 MPKI are very low. The slightly elevated L2 MPKI (2.39) with a high L2 miss *rate* (73.5%) is a cold-start artifact: the absolute number of L2 accesses is tiny (2,113), and most are compulsory misses during the initial loading of data into the cache hierarchy.
- At N=64, the 96 kB footprint exceeds L1D but fits comfortably in L2. IPC peaks at 2.13—the highest in our entire experiment set—because L1D misses are efficiently serviced by L2 (L2 MPKI is only 0.52) and the out-of-order core can exploit substantial instruction-level parallelism.
- At N=128, the 384 kB footprint exceeds the 256 kB L2 but only modestly: L2 MPKI remains low (0.24), suggesting that the kernel's access pattern still exhibits enough temporal locality to avoid catastrophic L2 thrashing. IPC remains healthy at 1.62.
- At N=192, the 864 kB footprint significantly exceeds L2. L2 MPKI rises to 17.27 and IPC degrades to 1.39, signaling the onset of memory-system pressure.
- At N=256, the 1.5 MB footprint overwhelms the 256 kB L2. L2 MPKI surges to 137.68 and IPC collapses to 0.45—a **severe throughput collapse** of nearly 4.7× versus the N=64 peak. Simulated execution time (0.181 s) is 266× longer than N=64 despite only a 56× increase in instruction count.

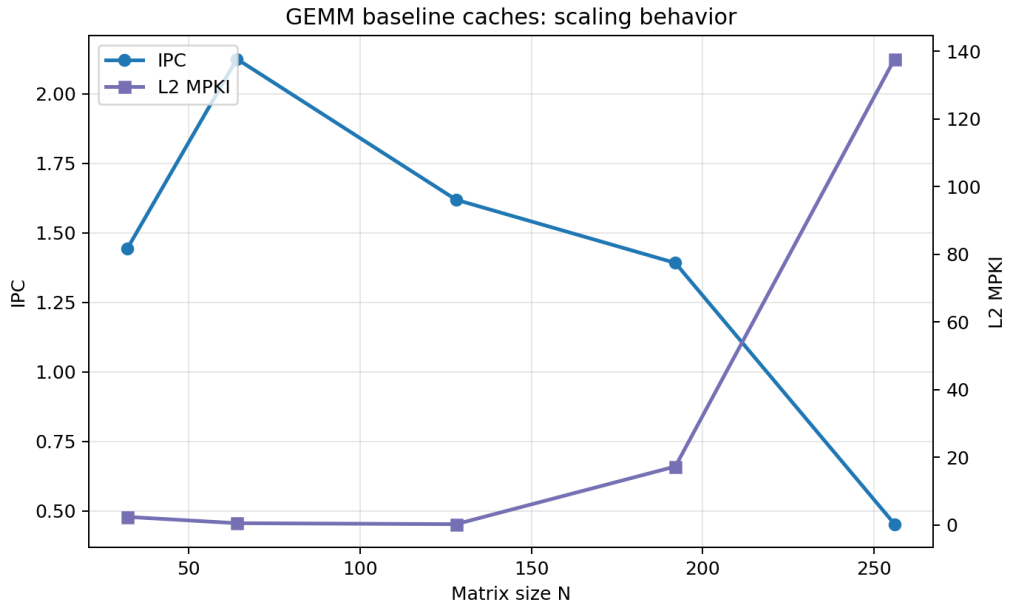


Figure 3: Baseline-cache scaling: IPC and L2 MPKI vs. problem size N (L1D=32 kB, L2=256 kB).

3.4 Bottleneck Attribution: Where Are the Cycles Going?

The scaling data above effectively functions as a CPI breakdown at the level of cache hierarchy. At N=64, CPI is 0.47—the core is compute-bound and the out-of-order engine is extracting substantial ILP. At N=256, CPI balloons to 2.21. The decomposition is:

- *Baseline compute cost* (from N=64, where caches are not the bottleneck): ≈0.47 CPI.
- *Memory-stall contribution* at N=256: $2.21 - 0.47 = 1.74$ CPI of additional stall — i.e., **over** 78% **of cycles at** N=256 **are attributable to the memory system**, dominated by L2-miss penalty (L2 MPKI of 137.68).

We summarize the evidence chain that identifies the primary performance bottleneck:

1. **L1D is not the dominant limiter at** N=128**.** Varying L1D across $\{16, 32, 64\}$ kB produces $< 0.01\%$ IPC variation (Table 2). L1D MPKI is consistently high (≈143–144), but these misses are serviced by L2.

2. **L2 capacity is a powerful lever.** At N=128, halving L2 from 256 kB to 128 kB collapses IPC by $> 2.2\times$ and raises L2 MPKI by $> 130\times$ (Table 3).

3. **Larger working sets amplify the L2 bottleneck.** At N=256 with baseline L2, IPC falls to 0.452 and L2 MPKI reaches 137.68 (Table 4).

4. **Why it occurs.** The *ijk* GEMM kernel streams through three $N \times N$ matrices. L1D cannot hold the active working set in any tested configuration (L1D MPKI is always high), so L2 becomes the *effective working-set container*. When the combined footprint exceeds L2 capacity, the processor stalls on off-chip (or next-level) memory accesses, manifesting as high L2 MPKI and low IPC. The high L1D miss rate is a necessary consequence of the streaming access pattern; what determines throughput is whether L2 can absorb those misses.

These results directly confirm our hypothesis; Once the working set exceeds L2 capacity, performance becomes dominated by L2 miss behavior rather than L1D effects.

4. Design Exploration and Optimization

Having identified L2 capacity as the dominant bottleneck, we propose and evaluate a mitigation: **enlarging the L2 cache at the stressed N=256 operating point**.

4.1 Architectural Change Explored

We hold L1D fixed at 32 kB (shown to be non-critical in Section 3) and sweep L2 over {256 kB, 512 kB, 1 MB, 2 MB} at N=256. Based on the bottleneck analysis, we expect a sharp improvement at the L2 size that first contains the 1.5 MB working set, with diminishing returns thereafter.

The 2 MB configuration was interrupted mid-run once; it was completed using `experiments/l2_fix_n256_2mb_resume.json`, with summary `experiments/logs/o3_run_20260421T215803Z_summary.json` and completed output under `results/gemm/n256_l1d32_l2_2MB/`.

4.2 Performance Improvements

Table 5: L2 remediation results at N=256, L1D=32 kB.

L2 size	IPC	CPI	L1D MPKI	L2 MPKI	simSeconds	Speedup
256 kB	0.451756	2.213586	141.9651	137.6844	0.180780	1.000
512 kB	0.524641	1.906064	136.9670	24.4104	0.155665	1.161
1 MB	1.680753	0.594972	143.4069	0.0944	0.048590	3.720
2 MB	1.683716	0.593924	143.3012	0.0154	0.048505	3.726

Figure 4 visualizes the remediation. The results are striking:

- **256 kB → 512 kB:** L2 MPKI drops significantly (from 137.68 to 24.41) but remains high enough to limit IPC to only 0.52—a modest 1.16× speedup. The working set still does not fit.
- **512 kB → 1 MB:** This is the **critical transition**. L2 MPKI plummets from 24.41 to 0.094, and IPC jumps from 0.52 to 1.68 (3.72× over baseline). Although the 1.5 MB working set does not fully fit in L2, it fits sufficiently well given the access pattern and associativity to avoid frequent conflict and capacity misses, allowing the out-of-order core to resume efficient execution.
- **1 MB → 2 MB:** IPC improves by only 0.18% (from 1.6808 to 1.6837). L2 MPKI falls further (0.094 → 0.015) but the absolute miss count is already so small that the performance impact is negligible. This confirms **diminishing returns beyond 1 MB**.

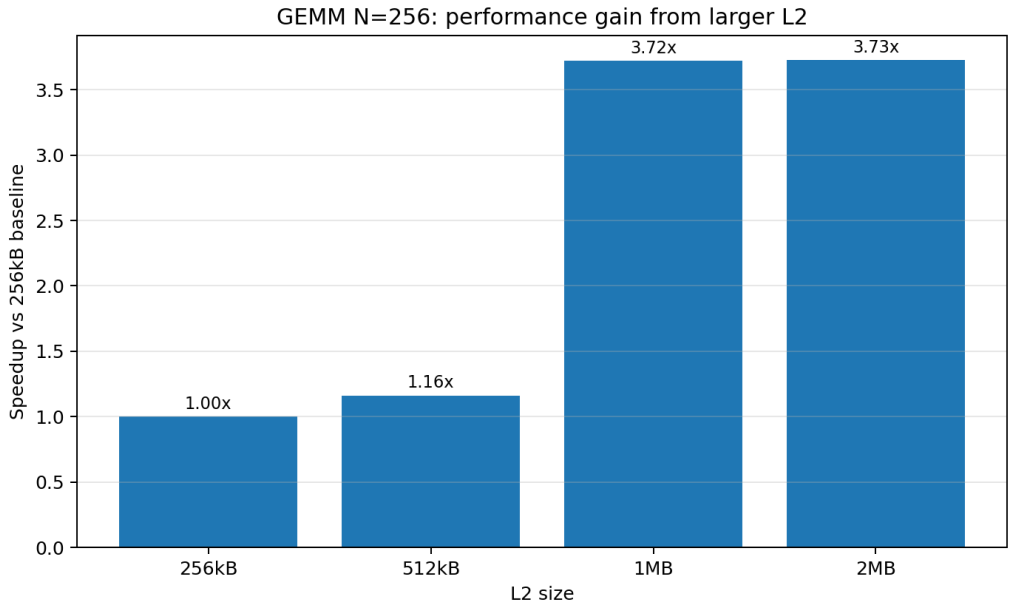


Figure 4: L2 remediation at N=256: performance and miss behavior vs. L2 size (combined panel; see Table 5 for exact values).

4.3 Trade-offs: Performance vs. Capacity, Energy, and Complexity

Table 6 restates the remediation data with simulated time, speedup vs. baseline, and an explicit capacity scaling column.

Table 6: Cost–benefit summary for L2 remediation at N=256 (baseline L2=256 kB). Speedups use simulated time (`simSeconds`).

L2 size	IPC	simSeconds	Speedup	Cap.	Interpretation
256 kB	0.451 756	0.180 780	1.000	1×	Reference
512 kB	0.524 641	0.155 665	1.161	2×	Small gain for 2× capacity
1 MB	1.680 753	0.048 590	3.720	4×	Large gain; bottleneck relief
2 MB	1.683 716	0.048 505	3.726	8×	≈0.2% extra vs. 1 MB

From 256 kB to 1 MB, speedup is ≈3.72×; from 1 MB to 2 MB, incremental speedup is only ≈1.002× (≈0.2%), effectively saturated.

Hardware (Area) Cost

Increasing L2 capacity enlarges SRAM tag and data arrays (area and subsystem complexity). In the context of our measured results:

- **256 kB → 512 kB (2× capacity):** delivers only a 1.16× speedup. L2 MPKI drops substantially (from 137.68 to 24.41) but remains high enough that the processor is still memory-bound. The capacity investment is partially effective but insufficient.

- **256 kB → 1 MB (4× capacity):** delivers a 3.72× speedup. The capacity investment directly removes the diagnosed bottleneck, bringing CPI from 2.21 down to 0.59.
- **1 MB → 2 MB (8× baseline capacity):** adds only ≈0.2% additional speedup. The extra silicon area and leakage power are difficult to justify for this workload point.

Energy / Power Considerations

Larger caches increase both static leakage and per-access dynamic energy. Given that the 1 MB → 2 MB transition produces negligible throughput gain, the larger cache is unlikely to be energy-efficient per unit of performance improvement.

Design Complexity Considerations

Larger caches introduce additional timing closure, verification, and floorplanning challenges in real hardware implementations. Since the 2 MB configuration adds substantial capacity with negligible measured throughput gain for this workload, its design cost is difficult to justify relative to the 1 MB alternative.

Conclusion of Cost–Benefit Analysis

The best measured tradeoff point is **L2=1** MB at N=256: it captures the majority of the recoverable performance (3.72× speedup, L2 MPKI reduced from 137.68 to 0.094, CPI reduced from 2.21 to 0.59) at 4× the baseline L2 capacity without the unnecessary energy and area expenditure of the 2 MB configuration. Going beyond 1 MB yields diminishing returns and represents a poor cost–benefit exchange for this workload.

5. Key Insights and Takeaways

Using detailed `O3CPU` simulation in gem5, we conducted a systematic study of cache hierarchy sensitivity for a dense GEMM workload and arrived at the following principal findings:

1. **GEMM transitions from compute-efficient to memory-system-limited behavior as the working set grows.** IPC peaks at 2.13 (N=64, where the working set fits in L2 and L1D misses are served as low-latency L2 hits, allowing the out-of-order core to overlap memory accesses and exploit high instruction-level parallelism) and collapses to 0.45 by N=256 (where the 1.5 MB footprint overwhelms the 256 kB L2).
2. **L2-capacity bottleneck is clearly identifiable.** At N=128, reducing L2 to 128 kB collapses IPC by $> 2.2\times$ and raises L2 MPKI by $> 130\times$. At N=256 with a 256 kB L2, IPC is only 0.45 with L2 MPKI of 138.
3. **L2 remediation is effective.** Increasing L2 from 256 kB to 1 MB at N=256 yields a 3.72× speedup, reduces CPI from 2.21 to 0.59, and reduces L2 MPKI from 138 to 0.094.
4. **Diminishing returns beyond 1 MB.** Further growth from 1 MB to 2 MB adds only ≈0.2% performance, making 1 MB the practical cost–benefit knee point for this workload.
5. **L1D is secondary in the tested regime.** L1D MPKI is consistently high (≈143) due to the streaming access pattern, but these misses are absorbed by L2. Varying L1D from 16–64 kB at N=128 produces $< 0.01\%$ IPC variation.

6. **Methodological takeaway.** Sensitivity sweeps combined with problem-size scaling cleanly separate *capacity* effects from *access pattern* effects. A high L1D miss rate is not, by itself, evidence of an L1D bottleneck; what matters is whether the next level of the hierarchy can absorb those misses.

All final quantitative conclusions are based on `O3CPU` simulation, matching the requirement for detailed microarchitectural simulation.

6. Limitations and Future Work

Limitations

- **Single workload class.** All conclusions are scoped to dense GEMM (ijk order). Other access patterns (e.g., sparse algebra, graph traversal) may exhibit different bottleneck profiles.
- **Limited cache associativity/replacement exploration.** We varied cache *capacity* but held associativity and replacement policy constant. These factors could interact with the capacity effects reported here.
- **No quantitative power/area modeling.** Energy and area claims are qualitative. A McPAT or CACTI integration would strengthen the cost analysis.
- **Cold-start effects at small N.** The N=32 data point shows anomalous L2 miss-rate behavior (73.5%) due to compulsory misses dominating the very small number of total L2 accesses (2,113). This does not affect our main conclusions, which focus on larger N values.
- **Bottleneck statements are configuration-scoped.** The identified knee points depend on the specific N values and cache sizes tested.

Future Work

- Evaluate loop-reordered GEMM variants (ikj, jik, blocked/tiled) to quantify how access pattern interacts with the L2 capacity knee identified here.
- Integrate McPAT/CACTI to attach quantitative area, leakage, and dynamic-energy estimates to each L2 configuration, which would turn the qualitative cost discussion in Section 4 into a formal performance-per-watt analysis.
- Sweep L2 associativity and replacement policy at the identified knee to check whether a smaller, smarter L2 can match the 1 MB baseline configuration.
- Extend the study to a second workload class (e.g., sparse SpMV or a graph kernel) to test the generality of the "L2 as working-set container" framing.

7. References

[1] The gem5 Simulator, https://www.gem5.org/.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[3] Project artifacts (internal): `experiments/results_summary_o3.csv`, `experiments/final_o3_manifest.json`, `experiments/l2_fix_n256_manifest.json`, `experiments/l2_fix_n256_2mb_resume.json`.

[4] Session logs (internal): `experiments/logs/o3_run_20260421T184741Z_summary.json`, `experiments/logs/o3_run_20260421T215803Z_summary.json`, `experiments/logs/o3_run_20260421T195017Z_manifest_snapshot.json`.

8. AI Usage Disclosure

Tools used. The team used AI assistants (ChatGPT and Claude) during the course of this project.

How they were used.

- *Writing:* Drafting and editing report prose; improving clarity, paragraph structure, and LaTeX formatting.
- *Debugging:* Diagnosing gem5 configuration errors and Python driver-script issues during the experiment-automation phase.
- *Analysis support:* Brainstorming candidate interpretations of anomalous data points (e.g., the cold-start N=32 behavior) and sanity-checking our CPI-stack reasoning.

What was AI-assisted. Portions of the report narrative (Sections 1, 3, 4, and 5 in particular) were drafted with AI assistance and then edited by the team for accuracy. The LaTeX scaffolding, figure placement, and table formatting were refined with AI help. All numerical values, tables, and figures were generated from our own simulation runs.

Verification.

- Every numerical value in the tables was cross-checked against the corresponding row in `experiments/results_summary_o3.csv`, which was produced directly from gem5 output by our `scripts/summarize_o3_results.py`.
- All figure claims were verified against the underlying run logs under `results/`.
- AI-drafted prose was reviewed line-by-line by team members for technical accuracy; any claim not directly supported by our measurements was removed or rewritten.
- Every piece of code (simulation scripts, plotting code) was executed by the team and its output inspected; AI was not trusted to generate final numerical results.

Appendix A: Full Results Table

Table 7 presents the complete set of final experiment results extracted from the consolidated O3-only CSV. Duplicate and legacy test rows are excluded.

Table 7: Complete O3CPU experiment results.

Tag	Caches	IPC	CPI	L1D MPKI	L2 MPKI	simSeconds
Cache sensitivity at N=128						
n128_l1d16_l2_256	L1D=16k, L2=256k	1.6195	0.6175	144.03	0.23	0.006575
n128_baseline	L1D=32k, L2=256k	1.6193	0.6175	143.87	0.24	0.006575
n128_l1d64_l2_256	L1D=64k, L2=256k	1.6194	0.6175	143.22	0.24	0.006575
n128_l1d32_l2_128	L1D=32k, L2=128k	0.7249	1.3794	124.04	31.79	0.014687
n128_l1d32_l2_512	L1D=32k, L2=512k	1.6188	0.6177	143.40	0.07	0.006577
n128_l1d32_l2_1MB	L1D=32k, L2=1M	1.6206	0.6171	143.34	0.05	0.006570
Working-set scaling (baseline caches)						
n32_baseline	L1D=32k, L2=256k	1.4430	0.6930	1.51	2.39	0.000150
n64_baseline	L1D=32k, L2=256k	2.1260	0.4704	43.92	0.52	0.000681
n128_baseline	L1D=32k, L2=256k	1.6193	0.6175	143.87	0.24	0.006575
n192_baseline	L1D=32k, L2=256k	1.3918	0.7185	151.87	17.27	0.025106
n256_baseline	L1D=32k, L2=256k	0.4518	2.2136	141.97	137.68	0.180780
L2 remediation at N=256						
n256_baseline	L1D=32k, L2=256k	0.4518	2.2136	141.97	137.68	0.180780
n256_l1d32_l2_512	L1D=32k, L2=512k	0.5246	1.9061	136.97	24.41	0.155665
n256_l1d32_l2_1MB	L1D=32k, L2=1M	1.6808	0.5950	143.41	0.09	0.048590
n256_l1d32_l2_2MB	L1D=32k, L2=2M	1.6837	0.5939	143.30	0.02	0.048505

Appendix B: Reproducibility

The complete experiment set can be reproduced with the following commands (paths relative to the project root):

```
# Main 10-run matrix
python3 scripts/run_o3_experiments.py \
    --manifest experiments/final_o3_manifest.json
# N=256 L2 remediation sweep
python3 scripts/run_o3_experiments.py \
    --manifest experiments/l2_fix_n256_manifest.json
# Resume single 2MB point (if interrupted)
python3 scripts/run_o3_experiments.py \
    --manifest experiments/l2_fix_n256_2mb_resume.json
# Rebuild consolidated CSV
python3 scripts/summarize_o3_results.py \
    --results_root results \
    --output_csv experiments/results_summary_o3.csv
```

Canonical artifacts.

- Consolidated O3 table: `experiments/results_summary_o3.csv`.
- Main matrix session `o3_run_20260421T184741Z`: `experiments/logs/o3_run_20260421T184741Z_summary.json`; manifest snapshot `experiments/logs/o3_run_20260421T184741Z_manifest_snapshot.json`.
- Remediation resume session `o3_run_20260421T215803Z`: `experiments/logs/o3_run_20260421T215803Z_summary.json`.
- Interrupted-run snapshot (historical): `experiments/logs/o3_run_20260421T195017Z_manifest_snapshot.json`.

Figure files for this PDF (copy into `plots/` next to `report.tex`, or symlink from `experiments/plots/`): `plots/01_n128_ipc_l2mpki_vs_l2.png`, `plots/02_n128_ipc_vs_l1d.png`, `plots/03_baseline_scaling_ipc_l2mpki_vs_n.png`, `plots/04_n256_l2_remediation_speedup.png`.

Appendix C: Figure-to-Claim Map

For drafting and cross-checking, each primary figure supports the following claims:

- `01_n128_ipc_l2mpki_vs_l2.png`**:** L2-capacity bottleneck at L2=128 kB vs. adequately sized L2 (N=128).
- `02_n128_ipc_vs_l1d.png`**:** L1D size is not the dominant limiter in the tested $\{16, 32, 64\}$ kB range (N=128, L2=256 kB).
- `03_baseline_scaling_ipc_l2mpki_vs_n.png`**:** Transition from compute-efficient behavior to memory-system pressure as N grows (baseline caches).
- `04_n256_l2_remediation_speedup.png`**:** L2 remediation at N=256; large gain from 256 kB to 1 MB, diminishing returns from 1 MB to 2 MB.